

Migration Plan: AutoERP + Global ERP Unification

Date: 2026-03-31 **Status:** Planning **Objective:** Merge AutoERP Workspace and Global ERP into a single unified system with the safest approach for database, structure, legacy CarGuru data, best UI/UX, and best performance.

Table of Contents

- 1. [Executive Summary](#)
- 2. [Overlap Analysis: What Global ERP Already Has](#)
- 3. [System Comparison Overview](#)
- 4. [Architecture Comparison](#)
- 5. [Database Comparison](#)
- 6. [UI/UX Comparison](#)
- 7. [Migration Strategy: Target Architecture](#)
- 8. [Database Migration Plan](#)
- 9. [Legacy CarGuru Data Safety Plan](#)
- 10. [Frontend Unification Plan](#)
- 11. [Backend Unification Plan](#)
- 12. [Performance Optimization Strategy](#)
- 13. [Risk Assessment & Mitigation](#)
- 14. [Migration Phases & Timeline](#)
- 15. [Rollback Strategy](#)

1. Executive Summary

Current State

Aspect	AutoERP Workspace	Global ERP
Purpose	Multi-tenant automotive SaaS (8 portals)	Single-app automotive ERP with advanced AI
Maturity	Feature-rich, broader scope	Production-ready, deeper operational depth
Database	Prisma ORM, ~97 models	Raw PostgreSQL, ~90+ tables, 186 migrations
Frontend	Next.js 14, React 18, 8 portal apps	Next.js 16, React 19, 1 unified app
Backend	7 Fastify microservices + Next.js API	Next.js API routes (441 endpoints)
AI	OpenAI only	OpenAI + Anthropic (dual provider)
Auth	JWT + OTP (Twilio)	Session cookies + JWT (mobile)
Styling	Tailwind + custom luxury design	Tailwind + 5 themes + RTL support

Decision: Global ERP as the Base System

Global ERP should be the target base for the following reasons:

- 1. **Newer tech stack** - Next.js 16, React 19 (vs Next.js 14, React 18)
- 2. **Production data** - Already contains live CarGuru/production data
- 3. **Deeper operational modules** - Call center, marketing with event tracking, PIS, RCC

- 4. **Better AI integration** - Dual provider (OpenAI + Anthropic), intelligence layer with 7 engines
- 5. **More mature DB schema** - 186 incremental migrations, structured columns over JSON blobs
- 6. **Simpler deployment** - Single app vs 8 portals + 7 microservices
- 7. **Better accounting** - Hierarchical COA with entities, journals, and journal lines
- 8. **Comprehensive HR** - Full payroll, allowances, visa management, gratuity calculation
- 9. **RTL/i18n support** - Arabic language and RTL layout built in
- 10. **Real-time integrations** - Dialer, call recordings, webhook infrastructure

What to Migrate FROM AutoERP (Revised After Overlap Analysis)

After thorough comparison, **Towing and RSA are already covered in Global ERP**. The actual migration scope is smaller than initially expected:

Feature	Priority	Status	Notes
Towing operations	HIGH	ALREADY EXISTS	Global ERP <code>recovery_requests</code> covers pickup/dropoff, agent dispatch, GPS, evidence capture, 5-stage workflow
RSA operations	HIGH	ALREADY EXISTS	Global ERP <code>leads</code> with <code>lead_type='rsa'</code> , 9-step workflow, <code>rsa_inspections</code> table, dispatch, mobile API, billing
Towing enhancements (truck fleet, route types)	MEDIUM	Enhance existing	Add truck management and CC/CW/WC/WW classification to <code>recovery_requests</code>
RSA enhancements (job type classification, response KPI)	MEDIUM	Enhance existing	Add job type enum and response time tracking to RSA leads
Rental & Leasing	MEDIUM	New module	Business expansion - no equivalent in Global ERP
Insurance & Warranty	MEDIUM	New module	Revenue stream - no equivalent in Global ERP
Workflow engine (configurable templates)	MEDIUM	New module	Global ERP has hardcoded flows; AutoERP has configurable workflow engine
DataTable component	HIGH	Port UI component	Superior unified table with sort/search/filter/pagination
Collapsible sidebar pattern	MEDIUM	Port UI pattern	Better navigation UX
Pre/Post service forms	LOW	Enhance existing	Global ERP already has pre-inspection forms; enhance with AutoERP patterns
Centralized booking system	MEDIUM	New module	Unified booking across all service types
Mobile apps (Expo)	MEDIUM	Port & adapt	Customer & technician mobile apps
WorkNet marketplace	LOW	Future feature	Inter-tenant bidding
Car Wash & EV Charging	LOW	Future feature	Ancillary services

2. Overlap Analysis: What Global ERP Already Has

TOWING / RECOVERY: Already Covered by **recovery_requests**

Global ERP's Recovery Requests (migrations 073-077, 156-157) is a full towing/recovery system:

Capability	Global ERP (recovery_requests)	AutoERP (TowJob)
Pickup/Dropoff workflow	Yes - type IN ('pickup', 'dropoff')	Yes - routeType IN ('CC', 'CW', 'WC', 'WW')
5-stage progression	New → Accepted → Reached → Picked Up → Dropped Off	Pending → Dispatched → En Route → Loading → In Transit → Delivered
Agent/driver dispatch	agent_name , agent_phone , agent_car_plate	driverId , truckId (FK references)
GPS locations	pickup_location , dropoff_location	pickupLat/Lng , dropoffLat/Lng
Evidence capture	Video + 5-angle photos (front, rear, left, right, cluster) for BOTH pickup and dropoff	Basic photo support
Terms confirmation	pickup_terms_shared_at , pickup_terms_confirmed_at	Not present
Pre-inspection form gating	Yes - blocks workflow until form submitted	Via workflow steps
Call center dashboard	3 buckets: pre_pickup, work_progress, happiness_check	Not present
Mobile API	Yes - /api/mobile/company/[companyId]/recovery-requests/*	Via RSA portal
Lead integration	Direct lead_id FK, auto-updates lead status to 'car_in'	Separate booking link
Verification & costing	verification_cost , verification_sale , verified_at	Via invoice metadata
Multi-channel comms	SMS, WhatsApp, Email for form sharing	Via notification service

What AutoERP adds that Global ERP lacks (enhancement only):

- Route type classification (CC=Customer→Customer, CW=Customer→Workshop, WC=Workshop→Customer, WW=Workshop→Workshop)
- Truck fleet management (truck types: flatbed, wheel_lift, heavy_recovery; capacity tracking)
- Distance/ETA tracking (distance_km, estimated_arrival)
- Vehicle storage tracking post-tow

Recommendation: Enhance **recovery_requests** with optional fields, NOT create new tables.

RSA (Roadside Assistance): Already Covered by Lead System

Global ERP's RSA (leads with **lead_type='rsa'**, migrations 011, 149-153) is a full RSA system:

Capability	Global ERP (leads + rsa_inspections)	AutoERP (RsaJob + RsaCall)
Lead type	lead_type = 'rsa' in leads table	Separate RsaJob model
9-step workflow	Accept → Start → Reach → Pre-Service Form → Inspection → Job Started → Complete → Post-Service → Close	Pending → Dispatched → En Route → On Scene → Resolved → Completed
Dispatch	POST /rsa/leads/[leadId]/dispatch with assignedUserId, branchId, remarks	dispatchAgent() function
RSA Inspections	Dedicated rsa_inspections table: VIN, health checks, battery voltage, starter, OBD codes, media	Via mini-inspection workflow step
Billing	Auto-creates estimate + paid invoice on "Complete Job" step	completeRsaJobWithBilling() - same concept
Mobile API	/api/v1/rsa/leads/assigned, /parts/quotes, /earnings/*	Portal-based pages
Earnings tracking	Inspection earnings + job card earnings + vendor parts quotes	Via invoice metadata
Status normalization	pending / done / lost (3 final states)	7 status states
Call center integration	Inquiry → RSA lead conversion via AI workflow	RsaCall with transcript
Recovery branching	RSA lead can branch to separate recovery lead	RSA can branch to towing workflow
Documentation	docs/rsa-inquiry-to-close-lead-flow.md (464 lines)	Inline code comments

RSA Stages in Global ERP (from jobFlows.ts):

```
new → dispatched → accepted → enroute → job_started → completed → closed/lost
```

What AutoERP adds that Global ERP lacks (enhancement only):

- Job type classification (FLAT_TYRE, BATTERY_JUMP, BATTERY_REPLACE, FUEL_DELIVERY, LOCKOUT, TOWING_REQUESTED, ACCIDENT, BREAKDOWN)
- Response time KPI tracking (dispatchedAt → onSceneAt = responseTimeMinutes)
- Insurance coverage flag (insurancePolicyRef, insuranceCovered)
- Priority levels (NORMAL, HIGH, URGENT)

Recommendation: Add optional fields to leads/rsa_inspections, NOT create new tables.

Summary: Overlap Impact on Migration Scope

ORIGINAL SCOPE (before analysis):		
└─ Towing (TowJob, TowTruck, TowDriver)	→ NEW TABLES	✗ NOT NEEDED
└─ RSA (RsaJob, RsaCall)	→ NEW TABLES	✗ NOT NEEDED
└─ Rental & Leasing	→ NEW TABLES	✓ Still needed

└─ Insurance & Warranty	→ NEW TABLES	☑ Still needed
└─ Workflow Engine	→ NEW TABLES	☑ Still needed
└─ UI Components (DataTable, Sidebar)	→ PORT CODE	☑ Still needed
└─ Mobile Apps	→ PORT CODE	☑ Still needed
└─ Enhancements to Towing/RSA	→ ALTER TABLES	☑ Smaller scope
REVISED SCOPE:		
└─ ENHANCE recovery_requests (add route_type, truck fields)	→ 1 migration	
└─ ENHANCE leads (add rsa_job_type, priority, response_time)	→ 1 migration	
└─ NEW: Truck fleet management table	→ 1 migration	
└─ NEW: Rental & Leasing tables	→ 1 migration	
└─ NEW: Insurance & Warranty tables	→ 1 migration	
└─ NEW: Workflow engine tables	→ 1 migration	
└─ NEW: Centralized bookings table	→ 1 migration	
└─ PORT: DataTable, Sidebar, animations	→ Code changes	
└─ PORT: Mobile apps (Expo)	→ New app	

Migration reduced from ~13 new tables to ~8 new tables + 3 ALTER TABLE enhancements.

3. System Comparison Overview

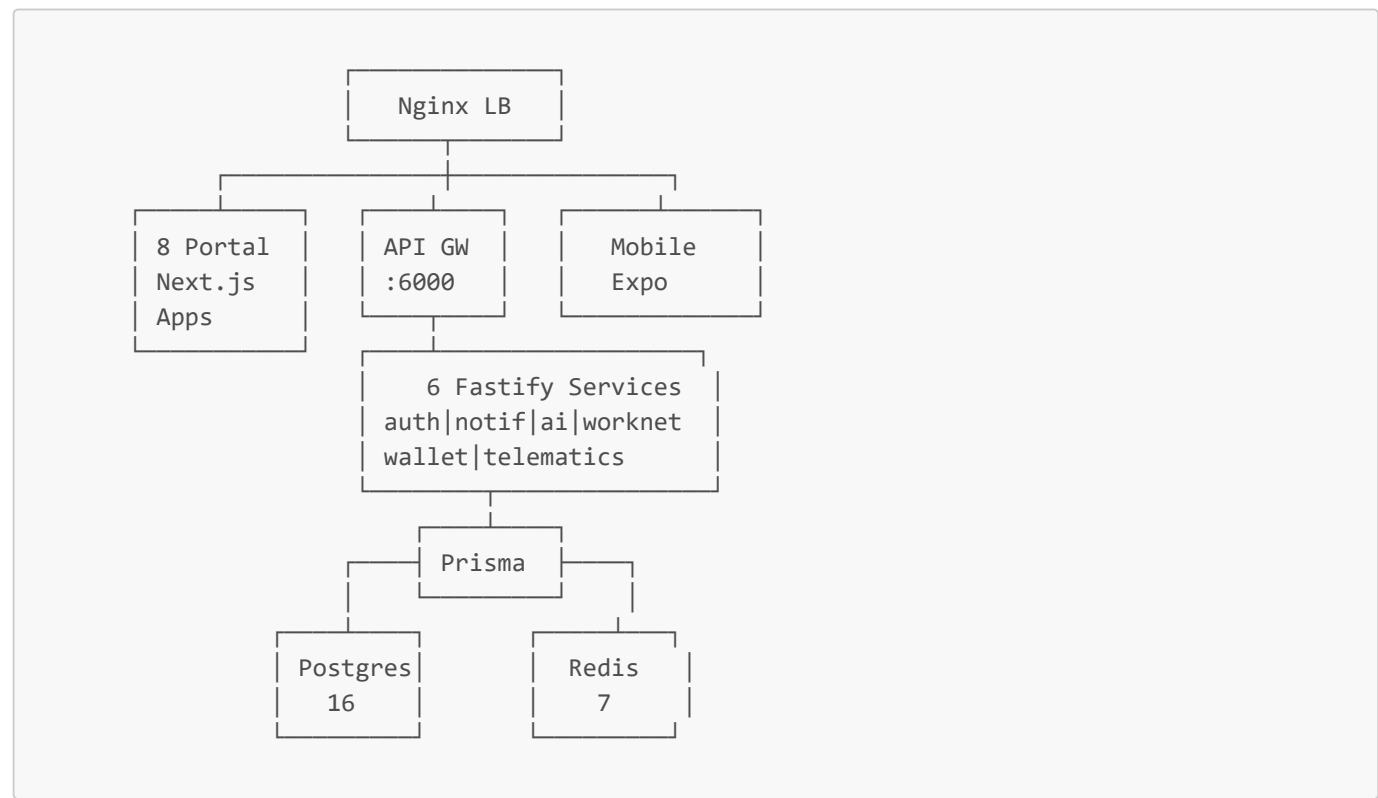
Technology Stack Side-by-Side

Layer	AutoERP	Global ERP	Winner	Notes
Runtime	Node.js 22+	Node.js 20 LTS	Global ERP	LTS = production stability
Framework	Next.js 14.2	Next.js 16.0	Global ERP	Latest framework
React	React 18.3	React 19.2	Global ERP	Server components, use() hook
TypeScript	5.6	5.9.2	Global ERP	Latest TS features
Database Client	Prisma 6.0	postgres 3.4 (raw)	Global ERP	Raw SQL = full control, better perf
Package Manager	npm + Turborepo	pnpm + Turborepo	Global ERP	pnpm = faster, stricter
Backend	Fastify microservices	Next.js API routes	Global ERP	Simpler ops, fewer moving parts
AI	OpenAI only	OpenAI + Anthropic	Global ERP	Multi-provider flexibility
Auth	JWT + OTP	Session + JWT	Global ERP	More secure (session cookies)
UI Icons	Lucide React	FontAwesome 7	Tie	Both good
Charts	None apparent	Recharts 3.8	Global ERP	Built-in charting

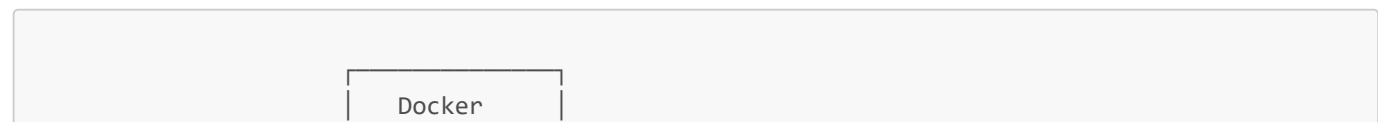
Layer	AutoERP	Global ERP	Winner	Notes
Forms	Custom FormField	react-hook-form	Global ERP	Production-proven
Validation	None apparent	Zod 3.23	Global ERP	Schema validation
Notifications	None apparent	Sonner 1.7	Global ERP	Toast system
i18n	next-intl 3.26	Custom + RTL	Global ERP	Arabic/RTL support
Theming	Light/Dark toggle	5 themes + RTL	Global ERP	Richer theming
Data Tables	Unified DataTable	Per-domain tables	AutoERP	Better reusability
Sidebar	Collapsible/pinnable	Fixed categories	AutoERP	Better UX
Mobile	Expo React Native	None	AutoERP	Mobile-ready
Multi-tenancy	tenant_id on all tables	company_id scoping	Global ERP	Production-proven

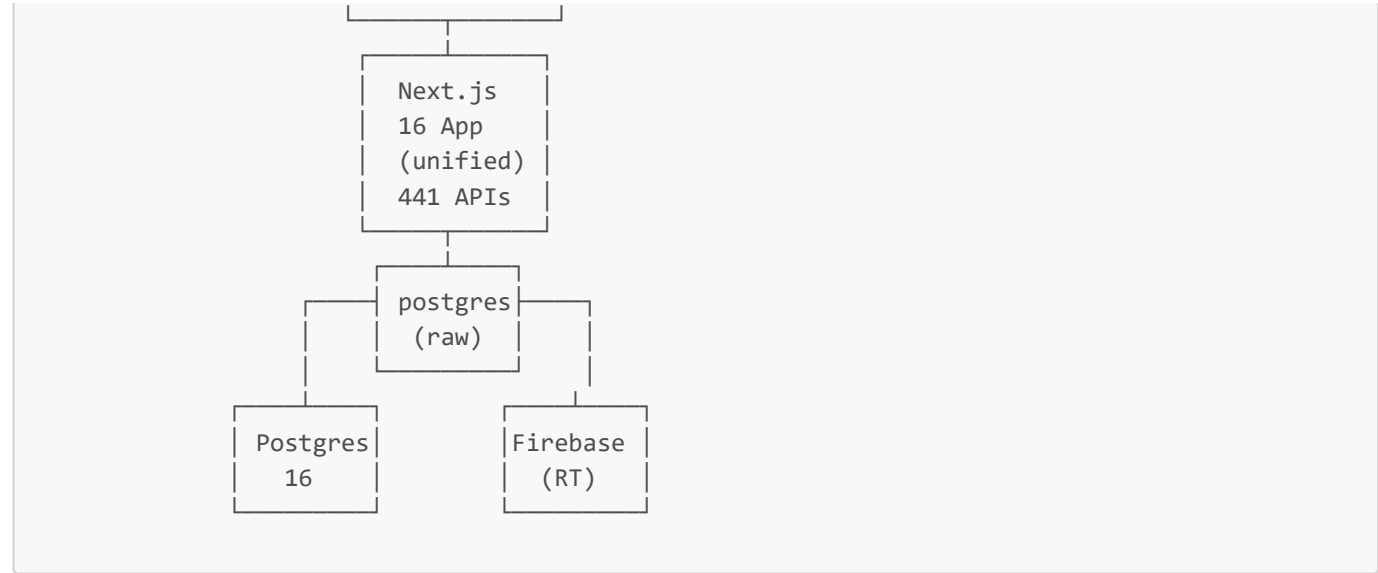
4. Architecture Comparison

AutoERP Architecture

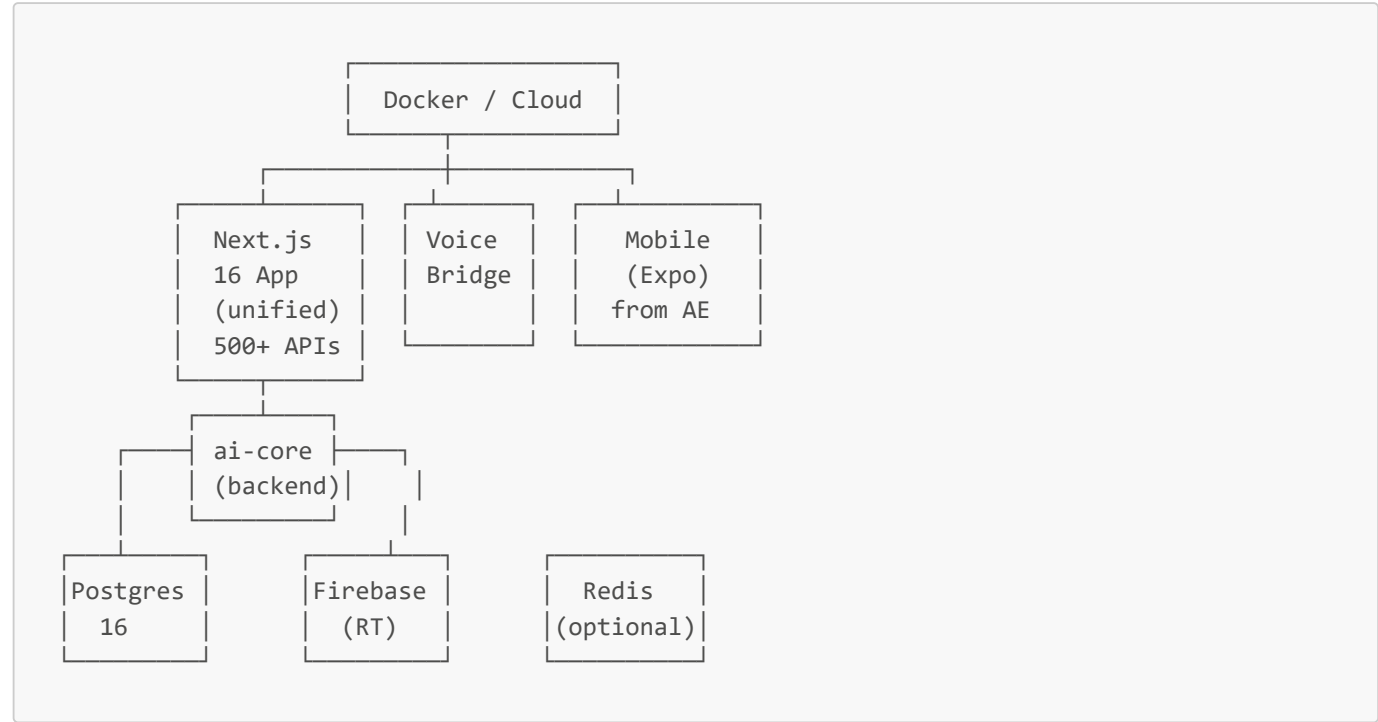


Global ERP Architecture (Current)





Target Architecture (Unified)



5. Database Comparison

Overlapping Entities (Exist in Both - Already Merged via Global ERP)

Entity	AutoERP Model	Global ERP Table	Verdict
Customer	Customer (tenant-based, wallet, loyalty tier)	customers (company-based, national_id, tax_number, multiple phones)	Keep Global ERP, enhance with loyalty fields
Vehicle	Vehicle (VIN, specs as JSON)	cars (structured fields, registration, insurance info)	Keep Global ERP, add specs JSONB
Work Orders	WorkOrder (self-contained, mileage, fils pricing)	work_orders (requires estimate_id, timing fields)	Keep Global ERP (enforces pipeline)

Entity	AutoERP Model	Global ERP Table	Verdict
Inspections	Inspection (JSON health scores, OBD data)	inspections (structured health fields, AI summaries)	Keep Global ERP (more queryable)
Estimates	Estimate (independent, approval token)	estimates (requires inspection_id, detailed pricing)	Keep Global ERP (enforces relationships)
Invoices	Invoice (simple, optional refs)	invoices (requires work_order_id, upstream refs)	Keep Global ERP (enforces traceability)
Leads	Lead (pipeline stages)	leads (lead_type, SLA tracking, sentiment scoring)	Keep Global ERP (more analytical)
Employees	Employee (basic HR fields)	employees (full payroll, visa, documents)	Keep Global ERP (far more comprehensive)
Accounting	Account + JournalEntry (flat)	accounting_accounts + journals + journal_lines (hierarchical)	Keep Global ERP (proper double-entry)
Inventory	InventoryItem (single table)	parts_catalog + inventory_stock + inventory_movements (normalized)	Keep Global ERP (better normalized)
Towing	TowJob , TowTruck , TowDriver	recovery_requests (full pickup/dropoff/agent/GPS/evidence)	ALREADY EXISTS - enhance only
RSA	RsaJob , RsaCall	leads (type='rsa') + rsa_inspections + 9-step workflow	ALREADY EXISTS - enhance only
Quality Checks	QualityCheck	quality_checks (multi-entity references)	Keep Global ERP
Roles/RBAC	Role with permissions	roles + permissions + role_permissions (normalized)	Keep Global ERP

Tables Only in AutoERP (to Migrate - Revised)

NEW Tables Needed (reduced scope):

- **tow_trucks** - Truck fleet management (types, capacity) - NOT in Global ERP
- **rental_vehicles**, **rental_bookings** - Rental operations
- **lease_contracts** - Leasing
- **insurance_policies**, **insurance_claims** - Insurance
- **warranty_policies**, **warranty_claims** - Warranty management
- **workflow_templates**, **workflow_instances** - Configurable workflow engine
- **bookings** - Centralized booking system
- **service_forms** - Pre/post service forms with tokenized URLs

ENHANCEMENTS to Existing Tables (ALTER TABLE only):

- **recovery_requests** - Add route_type, truck fields, distance/ETA
- **leads** (RSA) - Add rsa_job_type, priority, response_time_minutes
- **customers** - Add loyalty_tier, loyalty_points, preferred_language
- **cars** - Add specs JSONB, fuel_type, transmission
- **inspections** - Add obd_data JSONB, signature timestamps

Tables Only in Global ERP (All Retained)

All existing Global ERP tables are retained as-is. Key unique modules:

- Call center infrastructure (call_sessions, calls, call_recordings)
- Integration framework (channels, dialers, webhooks, health monitoring)
- Marketing with event tracking (email, SMS, WhatsApp, push events)
- PIS system (advisor scores, lead distribution, commission)
- Revenue Command Center (RCC)
- AI intelligence layer (global config, modules, feature toggles, signals)
- Billing system (invoices, profiles)
- User activity/risk/change tracking
- Comprehensive employee/HR tables with payroll

6. UI/UX Comparison

Design Philosophy

Aspect	AutoERP	Global ERP	Unified Target
Aesthetic	Luxury, clean, rounded-2xl	Modern neon, glowing gradients	Best of both: clean layout + rich theming
Sidebar	Collapsible, pinnable, hover-expand	Fixed with categories	Adopt AutoERP collapsible pattern
Data Tables	Unified DataTable (sorting, search, filter, pagination)	Per-domain tables	Adopt AutoERP DataTable
Forms	Custom FormField	react-hook-form + Zod	Keep Global ERP (react-hook-form + Zod)
Theming	Light/Dark toggle	5 themes + RTL	Keep Global ERP themes + RTL
Icons	Lucide React	FontAwesome 7	Standardize on FontAwesome (broader set)
Toasts	None	Sonner	Keep Sonner
Charts	None	Recharts	Keep Recharts
Animations	Staggered fade, slide	Fade/slide effects	Adopt AutoERP staggered animations
Accessibility	44px min button height, focus-visible	Basic	Adopt AutoERP accessibility standards

UI Components to Port from AutoERP

1. **DataTable** - Unified table with built-in sort/search/filter/pagination (604 lines)
2. **Collapsible Sidebar** - Pinnable, hover-expand, 64px/256px transition
3. **Badge variants** - More comprehensive badge system
4. **Loading states** - Skeleton patterns
5. **Staggered animations** - List item animations (50ms delays)

UI Features to Keep from Global ERP

1. **5-theme system** (midnight, sunset, ocean, forest, light) with RTL support

- 2. **react-hook-form + Zod** validation
- 3. **Recharts** dashboards
- 4. **Sonner** notifications
- 5. **react-select** for complex selectors
- 6. **Real-time call integration** (Linkus)
- 7. **Marketing campaign builder**
- 8. **AI control panel**
- 9. **Recovery request 5-step process UI** (927 lines)
- 10. **RSA dispatch and workflow UI**

7. Migration Strategy: Target Architecture

Principle: "Extend Global ERP, Don't Replace"

The unified system will:

- Use Global ERP as the foundation (code, DB, infrastructure)
- **Enhance existing tables** for towing/RSA with AutoERP's additional fields
- Add new tables only for genuinely missing modules (rental, insurance, workflow engine)
- Port AutoERP's superior UI components
- Preserve ALL legacy CarGuru data untouched

Target Tech Stack

Layer	Technology	Source
Runtime	Node.js 20 LTS	Global ERP
Framework	Next.js 16	Global ERP
React	React 19	Global ERP
TypeScript	5.9+	Global ERP
Database	PostgreSQL 16, raw postgres client	Global ERP
Package Manager	pnpm + Turborepo	Global ERP
Auth	Session cookies (web) + JWT (mobile)	Global ERP
AI	OpenAI + Anthropic (dual provider)	Global ERP
Forms	react-hook-form + Zod	Global ERP
UI Library	@repo/ui (enhanced with AutoERP components)	Merged
Charts	Recharts	Global ERP
Theming	5 themes + RTL + enhanced animations	Merged
Mobile	Expo (ported from AutoERP)	AutoERP

8. Database Migration Plan

Safety-First Principles

- 1. **NEVER modify existing Global ERP table structures** in Phase 1

2. **ADD new columns as nullable** only
3. **ADD new tables** with foreign keys to existing tables
4. **Full backup before every migration step**
5. **Reversible migrations** - every UP has a DOWN
6. **Test on clone database first**

Phase 1: Enhance Existing Tables (Migrations 187-189)

```
-- Migration 187: Enhance recovery_requests for AutoERP towing features
ALTER TABLE recovery_requests ADD COLUMN IF NOT EXISTS route_type VARCHAR(4)
CHECK (route_type IN ('CC', 'CW', 'WC', 'WW'));
ALTER TABLE recovery_requests ADD COLUMN IF NOT EXISTS truck_id UUID;
ALTER TABLE recovery_requests ADD COLUMN IF NOT EXISTS distance_km NUMERIC(8,2);
ALTER TABLE recovery_requests ADD COLUMN IF NOT EXISTS estimated_arrival TIMESTAMPTZ;
ALTER TABLE recovery_requests ADD COLUMN IF NOT EXISTS actual_arrival TIMESTAMPTZ;
ALTER TABLE recovery_requests ADD COLUMN IF NOT EXISTS vehicle_storage_needed BOOLEAN
DEFAULT false;

-- Migration 188: Enhance leads for RSA job type and KPI tracking
ALTER TABLE leads ADD COLUMN IF NOT EXISTS rsa_job_type VARCHAR(30)
CHECK (rsa_job_type IN ('flat_tyre', 'battery_jump', 'battery_replace',
'fuel_delivery', 'lockout', 'towing_requested', 'accident', 'breakdown'));
ALTER TABLE leads ADD COLUMN IF NOT EXISTS priority VARCHAR(10) DEFAULT 'normal'
CHECK (priority IN ('normal', 'high', 'urgent'));
ALTER TABLE leads ADD COLUMN IF NOT EXISTS response_time_minutes INT;
ALTER TABLE leads ADD COLUMN IF NOT EXISTS insurance_policy_ref VARCHAR(50);
ALTER TABLE leads ADD COLUMN IF NOT EXISTS insurance_covered BOOLEAN DEFAULT false;

-- Migration 189: Enhance customers and cars
ALTER TABLE customers ADD COLUMN IF NOT EXISTS loyalty_tier VARCHAR(20) DEFAULT
'standard';
ALTER TABLE customers ADD COLUMN IF NOT EXISTS loyalty_points INT DEFAULT 0;
ALTER TABLE customers ADD COLUMN IF NOT EXISTS preferred_language VARCHAR(5) DEFAULT
'en';
ALTER TABLE cars ADD COLUMN IF NOT EXISTS specs JSONB;
ALTER TABLE cars ADD COLUMN IF NOT EXISTS fuel_type VARCHAR(20);
ALTER TABLE cars ADD COLUMN IF NOT EXISTS transmission VARCHAR(20);
```

Phase 2: New Tables - Truck Fleet (Migration 190)

```
-- Migration 190: Tow Truck Fleet Management (new - not in Global ERP)
CREATE TABLE tow_trucks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  branch_id UUID REFERENCES branches(id),
  plate_number VARCHAR(20) NOT NULL,
  truck_type VARCHAR(20) NOT NULL CHECK (truck_type IN ('flatbed', 'wheel_lift',
'heavy_recovery')),
  capacity_kg NUMERIC(10,2),
  make VARCHAR(50),
  model VARCHAR(50),
  year INT,
  status VARCHAR(20) DEFAULT 'available'
CHECK (status IN ('available', 'dispatched', 'maintenance', 'retired')),
```

```

        assigned_driver_id UUID REFERENCES employees(id),
        created_at TIMESTAMPTZ DEFAULT NOW(),
        updated_at TIMESTAMPTZ DEFAULT NOW(),
        deleted_at TIMESTAMPTZ
    );

-- Add FK from recovery_requests to tow_trucks
ALTER TABLE recovery_requests ADD CONSTRAINT fk_recovery_truck
    FOREIGN KEY (truck_id) REFERENCES tow_trucks(id);

CREATE INDEX idx_tow_trucks_company ON tow_trucks(company_id);
CREATE INDEX idx_tow_trucks_status ON tow_trucks(status) WHERE deleted_at IS NULL;

```

Phase 3: New Tables - Rental & Leasing (Migration 191)

```

-- Migration 191: Rental & Leasing
CREATE TABLE rental_vehicles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    car_id UUID NOT NULL REFERENCES cars(id),
    daily_rate NUMERIC(10,2),
    weekly_rate NUMERIC(10,2),
    monthly_rate NUMERIC(10,2),
    status VARCHAR(20) DEFAULT 'available'
    CHECK (status IN ('available', 'rented', 'maintenance', 'retired')),
    mileage_limit_daily INT,
    excess_mileage_rate NUMERIC(8,2),
    insurance_included BOOLEAN DEFAULT false,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);

CREATE TABLE rental_bookings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    rental_vehicle_id UUID NOT NULL REFERENCES rental_vehicles(id),
    customer_id UUID NOT NULL REFERENCES customers(id),
    pickup_branch_id UUID REFERENCES branches(id),
    dropoff_branch_id UUID REFERENCES branches(id),
    start_date TIMESTAMPTZ NOT NULL,
    end_date TIMESTAMPTZ NOT NULL,
    actual_return_date TIMESTAMPTZ,
    status VARCHAR(20) DEFAULT 'reserved'
    CHECK (status IN ('reserved', 'active', 'returned', 'cancelled', 'overdue')),
    total_amount NUMERIC(12,2),
    deposit_amount NUMERIC(10,2),
    mileage_out INT,
    mileage_in INT,
    invoice_id UUID REFERENCES invoices(id),
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);

```

```

CREATE TABLE lease_contracts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  car_id UUID NOT NULL REFERENCES cars(id),
  customer_id UUID NOT NULL REFERENCES customers(id),
  monthly_payment NUMERIC(10,2),
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  total_value NUMERIC(14,2),
  residual_value NUMERIC(14,2),
  mileage_allowance INT,
  status VARCHAR(20) DEFAULT 'active'
  CHECK (status IN ('active', 'completed', 'terminated', 'defaulted')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);

CREATE INDEX idx_rental_vehicles_company ON rental_vehicles(company_id);
CREATE INDEX idx_rental_bookings_company ON rental_bookings(company_id);
CREATE INDEX idx_rental_bookings_dates ON rental_bookings(start_date, end_date);
CREATE INDEX idx_lease_contracts_company ON lease_contracts(company_id);

```

Phase 4: New Tables - Insurance & Warranty (Migration 192)

```

-- Migration 192: Insurance & Warranty
CREATE TABLE insurance_policies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  car_id UUID NOT NULL REFERENCES cars(id),
  customer_id UUID REFERENCES customers(id),
  provider VARCHAR(100),
  policy_number VARCHAR(50),
  coverage_type VARCHAR(30),
  premium NUMERIC(10,2),
  start_date DATE,
  end_date DATE,
  status VARCHAR(20) DEFAULT 'active'
  CHECK (status IN ('active', 'expired', 'cancelled', 'claimed')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);

CREATE TABLE insurance_claims (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  policy_id UUID NOT NULL REFERENCES insurance_policies(id),
  company_id UUID NOT NULL REFERENCES companies(id),
  claim_number VARCHAR(50),
  incident_date DATE,
  description TEXT,
  estimated_amount NUMERIC(12,2),
  approved_amount NUMERIC(12,2),
  status VARCHAR(20) DEFAULT 'submitted'
  CHECK (status IN ('submitted', 'under_review', 'approved', 'rejected', 'paid')),
  created_at TIMESTAMPTZ DEFAULT NOW(),

```

```

        updated_at TIMESTAMPTZ DEFAULT NOW(),
        deleted_at TIMESTAMPTZ
    );

CREATE TABLE warranty_policies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    car_id UUID NOT NULL REFERENCES cars(id),
    warranty_type VARCHAR(30),
    provider VARCHAR(100),
    coverage_details JSONB,
    start_date DATE,
    end_date DATE,
    max_claim_amount NUMERIC(12,2),
    status VARCHAR(20) DEFAULT 'active'
    CHECK (status IN ('active', 'expired', 'voided')),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE warranty_claims (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    warranty_id UUID NOT NULL REFERENCES warranty_policies(id),
    company_id UUID NOT NULL REFERENCES companies(id),
    work_order_id UUID REFERENCES work_orders(id),
    description TEXT,
    claim_amount NUMERIC(12,2),
    approved_amount NUMERIC(12,2),
    status VARCHAR(20) DEFAULT 'submitted'
    CHECK (status IN ('submitted', 'under_review', 'approved', 'rejected', 'paid')),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_insurance_policies_company ON insurance_policies(company_id);
CREATE INDEX idx_insurance_claims_policy ON insurance_claims(policy_id);
CREATE INDEX idx_warranty_policies_company ON warranty_policies(company_id);
CREATE INDEX idx_warranty_claims_warranty ON warranty_claims(warranty_id);

```

Phase 5: New Tables - Workflow Engine & Bookings (Migration 193)

```

-- Migration 193: Configurable Workflow Engine + Centralized Bookings
CREATE TABLE workflow_templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID REFERENCES companies(id),
    name VARCHAR(100) NOT NULL,
    flow_type VARCHAR(30) NOT NULL,
    steps JSONB NOT NULL DEFAULT '[]',
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE workflow_instances (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    template_id UUID NOT NULL REFERENCES workflow_templates(id),

```

```

    company_id UUID NOT NULL REFERENCES companies(id),
    entity_type VARCHAR(30) NOT NULL,
    entity_id UUID NOT NULL,
    current_step INT DEFAULT 0,
    status VARCHAR(20) DEFAULT 'active'
    CHECK (status IN ('active', 'completed', 'cancelled', 'paused')),
    step_data JSONB DEFAULT '{}',
    branched_from_id UUID REFERENCES workflow_instances(id),
    branched_to_id UUID REFERENCES workflow_instances(id),
    started_at TIMESTAMPTZ DEFAULT NOW(),
    completed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE bookings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    branch_id UUID REFERENCES branches(id),
    customer_id UUID NOT NULL REFERENCES customers(id),
    car_id UUID REFERENCES cars(id),
    booking_type VARCHAR(20) NOT NULL
    CHECK (booking_type IN ('workshop', 'towing', 'rsa', 'rental', 'inspection')),
    scheduled_date TIMESTAMPTZ NOT NULL,
    status VARCHAR(20) DEFAULT 'confirmed'
    CHECK (status IN ('pending', 'confirmed', 'in_progress', 'completed', 'cancelled',
'no_show')),
    notes TEXT,
    lead_id UUID REFERENCES leads(id),
    work_order_id UUID REFERENCES work_orders(id),
    recovery_request_id UUID REFERENCES recovery_requests(id),
    rental_booking_id UUID REFERENCES rental_bookings(id),
    workflow_instance_id UUID REFERENCES workflow_instances(id),
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    deleted_at TIMESTAMPTZ
);

CREATE INDEX idx_workflow_instances_entity ON workflow_instances(entity_type, entity_id);
CREATE INDEX idx_workflow_instances_status ON workflow_instances(status);
CREATE INDEX idx_bookings_company ON bookings(company_id);
CREATE INDEX idx_bookings_customer ON bookings(customer_id);
CREATE INDEX idx_bookings_date ON bookings(scheduled_date);
CREATE INDEX idx_bookings_type ON bookings(booking_type);

```

Phase 6: Permissions for New Modules (Migration 194)

```

-- Migration 194: Permissions for new modules
INSERT INTO permissions (key, label, module, scope) VALUES
-- Rental
('rental.view', 'View Rentals', 'rental', 'company'),
('rental.create', 'Create Rental Bookings', 'rental', 'company'),
('rental.edit', 'Edit Rental Bookings', 'rental', 'company'),
('rental.vehicles.manage', 'Manage Rental Fleet', 'rental', 'company'),
-- Insurance

```

```
('insurance.view', 'View Insurance Policies', 'insurance', 'company'),
('insurance.create', 'Create Policies', 'insurance', 'company'),
('insurance.claims', 'Manage Claims', 'insurance', 'company'),
-- Warranty
('warranty.view', 'View Warranties', 'warranty', 'company'),
('warranty.create', 'Create Warranties', 'warranty', 'company'),
('warranty.claims', 'Manage Warranty Claims', 'warranty', 'company'),
-- Tow Trucks
('tow_trucks.view', 'View Tow Trucks', 'recovery', 'company'),
('tow_trucks.manage', 'Manage Tow Truck Fleet', 'recovery', 'company'),
-- Bookings
('booking.view', 'View Bookings', 'booking', 'company'),
('booking.create', 'Create Bookings', 'booking', 'company'),
('booking.edit', 'Edit Bookings', 'booking', 'company'),
-- Workflow
('workflow.templates', 'Manage Workflow Templates', 'workflow', 'company'),
('workflow.view', 'View Workflows', 'workflow', 'company')
ON CONFLICT DO NOTHING;
```

9. Legacy CarGuru Data Safety Plan

Critical: Zero Data Loss Guarantee

LEGACY DATA SAFETY PROTOCOL
1. SNAPSHOT before any migration pg_dump -Fc global_erp > pre_migration.dump
2. NEVER ALTER existing column types Only ADD new columns (nullable)
3. NEVER DELETE existing rows Soft delete (deleted_at) only
4. NEVER RENAME existing columns Add new column + backfill + deprecate
5. TEST on clone database first pg_restore to test DB, run migrations there
6. VERIFY row counts after each migration SELECT count(*) from every table
7. KEEP backup for 90 days minimum

Pre-Migration Checklist

```
# 1. Full database backup
pg_dump -Fc -h $DB_HOST -U $DB_USER -d global_erp \
  -f backup_$(date +%Y%m%d_%H%M%S).dump

# 2. Record current table counts
psql -c "SELECT schemaname, relname, n_live_tup
        FROM pg_stat_user_tables
        ORDER BY relname;" > table_counts_before.txt

# 3. Clone to test database
createdb global_erp_migration_test
pg_restore -d global_erp_migration_test backup_latest.dump

# 4. Run ALL migrations on test DB first
DATABASE_URL=...migration_test pnpm db:migrate

# 5. Verify test DB integrity
psql -d global_erp_migration_test -c "
  SELECT schemaname, relname, n_live_tup
  FROM pg_stat_user_tables
  ORDER BY relname;" > table_counts_after_test.txt

# 6. Diff counts (should show ZERO changes to existing tables)
diff table_counts_before.txt table_counts_after_test.txt
```

Data That Must NOT Be Touched

Data Category	Tables	Action
All customer records	customers, customer_car_links, customer_wallet_transactions	READ ONLY during migration
All financial records	invoices, invoice_items, accounting_, <i>billing_</i>	READ ONLY during migration
All work history	work_orders, inspections, estimates, quality_checks	READ ONLY during migration
All call center data	calls, call_sessions, call_recordings	READ ONLY during migration
All employee/HR data	employees, employee_allowances, user_*	READ ONLY during migration
All lead/CRM data	leads, lead_events, lead_bookings	READ ONLY during migration
All recovery requests	recovery_requests	READ ONLY during migration (new columns are nullable)
All RSA data	rsa_inspections	READ ONLY during migration
All integration configs	integration_channels, integration_dialers	READ ONLY during migration

10. Frontend Unification Plan

Step 1: Port AutoERP DataTable to Global ERP

Port `packages/ui/src/components/DataTable.tsx` (604 lines) from AutoERP into Global ERP's `packages/ui/src/`:

Features to bring:

- Column sorting with visual indicators
- Column-level text search with popover
- Date range filtering (auto-detect)
- Configurable pagination [10, 25, 50, 100]
- Null-safe sorting
- Responsive design

Adapt to Global ERP:

- Use FontAwesome icons instead of Lucide
- Apply Global ERP theme tokens
- Support RTL layout

Step 2: Enhance Sidebar Navigation

Port AutoERP's collapsible sidebar pattern:

- 64px collapsed / 256px expanded
- Hover-expand with smooth transitions
- Pinnable state (localStorage persistence)
- Staggered animation for nav items

Merge with Global ERP's:

- Category-based navigation structure
- Company/branch context switcher
- Real-time call notification integration

Step 3: Add New Module Pages

```
apps/web/app/company/[companyId]/
├── rental/
│   ├── page.tsx                (available vehicles)
│   ├── bookings/page.tsx      (booking management)
│   └── [bookingId]/page.tsx    (booking detail)
├── insurance/
│   ├── policies/page.tsx      (policy list)
│   └── claims/page.tsx        (claims management)
├── warranty/
│   ├── policies/page.tsx      (warranty list)
│   └── claims/page.tsx        (claims management)
├── tow-trucks/
│   └── page.tsx                (truck fleet management)
└── bookings/
    └── page.tsx                (centralized booking calendar)
```

Corresponding API routes:

```
apps/web/app/api/company/[companyId]/
├── rental/
│   ├── vehicles/route.ts
│   └── bookings/route.ts
├── insurance/
│   ├── policies/route.ts
│   └── claims/route.ts
├── warranty/
│   ├── policies/route.ts
│   └── claims/route.ts
├── tow-trucks/route.ts
└── bookings/route.ts
```

11. Backend Unification Plan

New Backend Modules (in packages/ai-core/src/)

```
packages/ai-core/src/
├── rental/
│   ├── types.ts
│   ├── queries.ts
│   ├── actions.ts
│   └── index.ts
├── insurance/
│   ├── types.ts
│   ├── queries.ts
│   └── index.ts
├── warranty/
│   ├── types.ts
│   ├── queries.ts
│   └── index.ts
├── tow-trucks/
│   ├── types.ts
│   ├── queries.ts
│   └── index.ts
├── workflow/
│   ├── types.ts
│   ├── engine.ts
│   ├── templates.ts
│   └── index.ts
└── booking/
    ├── types.ts
    ├── queries.ts
    ├── actions.ts
    └── index.ts
```

(port from AutoERP, rewrite Prisma → raw SQL)

(port booking-to-job from AutoERP)

Key Adaptation: Prisma to Raw SQL

All code ported from AutoERP needs conversion from Prisma to raw **postgres** client:

```
// AutoERP (Prisma)
const vehicle = await db.rentalVehicle.create({
  data: { tenantId, carId, dailyRate: 15000, status: 'AVAILABLE' }
});

// Global ERP (raw postgres)
const [vehicle] = await sql`
  INSERT INTO rental_vehicles (company_id, car_id, daily_rate, status)
  VALUES (${companyId}, ${carId}, ${15000}, 'available')
  RETURNING *
`;
```

Enhance Existing Modules (No New Files Needed)

- **Recovery requests** - Add truck assignment logic to existing recovery actions
- **RSA leads** - Add job type and response time tracking to existing lead repository
- **Customers** - Add loyalty tier logic to existing customer queries
- These are field additions to existing queries, not new modules

12. Performance Optimization Strategy

Database Performance

Optimization	Description
Partial indexes	WHERE deleted_at IS NULL on all new tables
Composite indexes	(company_id, status) on all operational tables
Connection pooling	Already using postgres client's built-in pooling
Raw SQL advantage	No ORM overhead (Global ERP's existing approach)
Materialized views	For dashboard aggregations (RCC, PIS, booking calendar)

Frontend Performance

Optimization	Description
Server Components	Default to server rendering (already in Global ERP)
React 19 Streaming	Suspense streaming for heavy pages
Dynamic imports	Lazy-load new modules (rental, insurance, booking calendar)
Virtualized DataTable	Add virtualization for 1000+ row tables
Image optimization	Sharp + Next.js Image (already in Global ERP)

API Performance

Optimization	Description
Cursor pagination	For large datasets in new modules
Field selection	Only SELECT needed columns

Optimization	Description
Batch operations	Bulk insert/update for imports
Redis caching	Optional: for session store, rate limiting

13. Risk Assessment & Mitigation

Risk	Probability	Impact	Mitigation
Data loss during migration	LOW	CRITICAL	Full backup, test on clone, row count verification
Breaking existing features	LOW	HIGH	Only ADD columns/tables, never modify existing
Performance regression	LOW	MEDIUM	New tables have proper indexes from day 1
Schema conflicts	LOW	HIGH	Only nullable ALTER TABLE ADD COLUMN
Auth/permission gaps	MEDIUM	MEDIUM	New modules use existing RBAC + new permissions
UI inconsistency	MEDIUM	LOW	Port DataTable as unified component

14. Migration Phases & Timeline

Phase 1: Database Enhancements (Migrations 187-189)

- ☐ Full database backup
- ☐ Clone to test database
- ☐ Run enhancement migrations (ALTER TABLE ADD COLUMN)
- ☐ Verify zero impact on existing data
- ☐ Deploy to production

Phase 2: New Tables (Migrations 190-194)

- ☐ Create tow_trucks table
- ☐ Create rental & leasing tables
- ☐ Create insurance & warranty tables
- ☐ Create workflow engine tables
- ☐ Create bookings table
- ☐ Add permissions
- ☐ Test on clone, deploy to production

Phase 3: UI Enhancement

- ☐ Port DataTable component from AutoERP
- ☐ Enhance sidebar with collapsible pattern
- ☐ Add staggered animations and accessibility improvements
- ☐ Update sidebar config with new module entries

Phase 4: Backend Modules

- ☐ Implement rental backend (queries, actions, API routes)
- ☐ Implement insurance/warranty backend
- ☐ Implement tow truck fleet management
- ☐ Port workflow engine from AutoERP (Prisma → raw SQL)

- ☐ Implement centralized booking system
- ☐ Create UI pages for all new modules

Phase 5: Mobile App Integration

- ☐ Port Expo mobile app from AutoERP
- ☐ Adapt mobile API endpoints to Global ERP auth (JWT)
- ☐ Test mobile flows end-to-end

Phase 6: Performance & Cleanup

- ☐ Add materialized views for dashboards
- ☐ Performance audit
- ☐ Documentation update

15. Rollback Strategy

Each phase is independently reversible:

Phase	Rollback Method
Phase 1 (ALTER TABLE)	<code>ALTER TABLE DROP COLUMN</code> for each added column
Phase 2 (New Tables)	<code>DROP TABLE</code> for new tables only
Phase 3 (UI)	Revert git commits
Phase 4 (Backend)	Revert git commits, disable routes
Phase 5 (Mobile)	Unpublish apps
Phase 6 (Perf)	Drop materialized views

No point of no return - all existing data is untouched throughout.

Appendix A: AutoERP Features NOT Being Migrated (and Why)

AutoERP Feature	Reason Not Migrated
TowJob table	<code>recovery_requests</code> already covers this with richer workflow
RsaJob table	<code>leads</code> (type='rsa') + <code>rsa_inspections</code> already covers this
RsaCall table	Global ERP's <code>call_sessions</code> + <code>calls</code> already handles call logging
8 Portal apps	Global ERP's single unified app is simpler and more maintainable
7 Fastify microservices	Next.js API routes are sufficient, fewer moving parts
Prisma ORM	Raw SQL gives better performance and full PostgreSQL feature access
WorkNet marketplace	Future feature, not core business need now
Car Wash module	Low priority ancillary service
EV Charging module	Low priority ancillary service
Training/Certification	Not in scope

AutoERP Feature	Reason Not Migrated
Support Tickets	Not in scope
Tenant model	Global ERP uses companies (same concept, different naming)

Appendix B: File-by-File Migration Map

AutoERP Source	Global ERP Target	Action
packages/ui/src/components/DataTable.tsx	packages/ui/src/components/DataTable.tsx	Port + adapt to theme system
packages/ui/src/platform/PortalSidebar.tsx	Enhance apps/web/components/layout/	Extract collapsible pattern
packages/utils/src/workflow-engine.ts	packages/ai-core/src/workflow/engine.ts	Rewrite Prisma → raw SQL
packages/utils/src/booking-to-job.ts	packages/ai-core/src/booking/actions.ts	Rewrite Prisma → raw SQL
packages/utils/src/money.ts	packages/ai-core/src/utils/money.ts	Direct port (file conversion utility)
mobile/customer-app/	New apps/mobile/ or separate repo	Port with API adaptation
mobile/technician-app/	New apps/mobile-tech/ or separate repo	Port with API adaptation

Appendix C: Sidebar Config Update

Add to packages/ui/src/layout/sidebarConfig.ts:

```
// New module entries
{ label: 'Tow Trucks', icon: 'truck-tow', href: '/tow-trucks', module: 'recovery' },
{ label: 'Rental', icon: 'car-side', href: '/rental', module: 'rental' },
{ label: 'Insurance', icon: 'shield-check', href: '/insurance', module: 'insurance' },
{ label: 'Warranty', icon: 'certificate', href: '/warranty', module: 'warranty' },
{ label: 'Bookings', icon: 'calendar-check', href: '/bookings', module: 'booking' },
```

